

DOCUMENTAÇÃO TÉCNICA

DelBicos – Delivery de Bicos

1 IDENTIFICAÇÃO DO PROJETO

1.1. Nome do Projeto: DelBicos – Delivery de Bicos

1.2 Grupo/Equipe

- Douglas Wenzel, 3011392413022
- Eduardo Kamo, 3011392413005
- Fernando Chibli, 3011392323017
- Fernando Ribeiro, 3011392413020
- Gustavo Ferreira, 3011392413016
- Iago Rossan, 3011392413038
- Isabel Maito, 3011392413045
- Lucas Consani, 3011392413046

1.3. Data da Elaboração: 22/08/2026

1.4 Versão do Documento: 2.3 [24/09/2026]

2 VISÃO GERAL DO PRODUTO

2.1 Problema a ser Resolvido

Profissionais autônomos que prestam pequenos serviços domésticos e prediais (limpeza, pequenos reparos, montagem de móveis, elétrica etc. — os “bicos”) têm dificuldade para divulgar seu trabalho, organizar a agenda e comprovar renda de forma profissional. Do lado do cliente, falta um canal único e confiável para encontrar, comparar, agendar e pagar esses profissionais, com garantias mínimas de segurança (avaliações, pagamento intermediado, comunicação registrada).

2.2 Solução Proposta

O DelBicos é um marketplace multiplataforma (app mobile Android/iOS e versão web, mesmo código-base em React Native + Expo) que conecta clientes e profissionais autônomos. Oferece busca de profissionais por categoria/subcategoria e geolocalização (raio de atuação); perfil do profissional com portfólio, comodidades e avaliações; agendamento respeitando a disponibilidade real cadastrada; pagamento por cartão via Stripe, com o agendamento só sendo criado após a confirmação do pagamento; chat em tempo real (Socket.IO) por agendamento; notificações push (Expo); e um chatbot com Processamento de Linguagem Natural e comando de voz, já integrado na branch stag-facul, para agendar, consultar e cancelar serviços por texto ou fala.

2.3 Público-Alvo

Duas personas principais — (1) Cliente residencial/comercial que precisa contratar um “bico” pontual e quer rapidez, preço claro e segurança para deixar um desconhecido em sua casa; (2) Profissional autônomo ou pequena prestadora de serviços que quer divulgar seu trabalho, controlar a própria agenda e receber pagamentos sem burocracia. Uma terceira persona, o Administrador da plataforma, modera categorias e profissionais e acompanha métricas gerais (dashboard admin).

3 REQUISITOS FUNCIONAIS E NÃO FUNCIONAIS

3.1 Requisitos Funcionais (RFs)

RF01 Alternância de Perfil – trocar entre a visão Cliente e Parceiro por um switch no menu. RF02 Onboarding do Parceiro – cadastro de dados profissionais (especialidades, biografia, documentos). RF03 Gestão de Bicos – CRUD de serviços oferecidos pelo profissional. RF04 Filtro de Geolocalização – o parceiro define um raio de atuação (km) e só recebe pedidos dentro dele. RF05 Gestão de Status do Agendamento – fluxo Aceitar (confirmed) → A caminho (on the way) → Iniciar (in progress) → Concluir (completed). RF06 Agenda Dinâmica – visualização dos agendamentos confirmados em lista/calendário. RF07 Chat Contextual – canal de texto entre cliente e parceiro após a confirmação do agendamento. RF08 Check-out de Segurança – confirmação para finalizar o serviço. RF09 Portfólio (mídia) – upload de fotos dos trabalhos realizados. RF10 Relatórios de Ganhos – exportação do histórico de serviços e valores. Também já implementados: autenticação/cadastro com verificação de e-mail, favoritos, avaliações (rating + review), dashboard de KPIs do profissional e agendamento por PLN/voz via chatbot (branch stag-facul).

3.2 Requisitos Não Funcionais (RNFs)

RNF01 Responsividade – interface adaptável de smartphone a tablet (5" a 11") e à web. RNF02 Persistência Local – armazenamento local (AsyncStorage) para agenda e rascunhos offline. RNF03 Sincronização – dados locais sobem para o backend assim que há conexão. RNF04 Segurança JWT – controle de acesso por papel (Cliente, Parceiro, Admin) em todos os endpoints autenticados. RNF05 Performance de Mídia – compressão de imagens antes do upload. RNF06 Validação Robusta – validação no front (React Hook Form) e no backend (middlewares Express). RNF07 Sincronismo em Nuvem – alterações no mobile devem refletir no backend em menos de 2s (REST + Socket.IO). RNF08 Acessibilidade – suporte a VLibras e contraste no modo escuro/alto contraste. Cada RF está associado a pelo menos um RNF (ex.: RF05 depende de RNF07 para refletir status em tempo real nos dois lados).

4 ARQUITETURA DO SISTEMA

4.1 Visão Geral da Arquitetura

Arquitetura cliente-servidor com um backend único (monólito modular) em Node.js/Express expondo uma API REST, complementado por um serviço Python (nlp-service) dedicado a PLN/embeddings, consumido pela API Node. O frontend é um único app React Native (Expo) que compila para mobile (Android/iOS) e para web (React Native Web), consumindo a mesma API. Comunicação em tempo real (chat e status de agendamento) via Socket.IO no mesmo servidor HTTP; atualização do catálogo de serviços via Server-Sent Events (SSE). Bancos políglotas: PostgreSQL (Sequelize) para as entidades de negócio e MongoDB (Mongoose) para mensagens de chat e logs de login. Fluxo: App Mobile/Web $\rightleftharpoons$ REST/JWT + Socket.IO $\rightleftharpoons$ API Express/TypeScript $\rightleftharpoons$ {PostgreSQL, MongoDB, nlp-service, Stripe, SendGrid, S3/ImgBB, Expo Push}. Organização em camadas (branch refactor/estrutura-seguranca): no backend, rotas $\rightarrow$ controllers finos (apenas HTTP) $\rightarrow$ services por domínio (src/services/appointment, catalog, professional, auth, dashboard, notification), que concentram regras de negócio e autorização $\rightarrow$ models Sequelize; regras puras ficam em arquivos *.rules.ts, testáveis sem banco. Erros de domínio são lançados como HttpError e convertidos em resposta por um errorHandler global; a montagem do Express (src/app.ts) é separada da inicialização do servidor (server.ts). No frontend, as chamadas HTTP ficam em uma camada src/api/ por domínio (auth, payments, chatbot) sobre um único cliente Axios que injeta o JWT, e regras puras do chatbot ficam em src/lib/chatbot/.

4.2 Componentes Principais

Frontend – React Native 0.81 + Expo 54, TypeScript, Zustand 5 (estado), React Navigation 7, Axios, Stripe React Native/JS, React Native Maps. Backend – Node.js ≥ 18 , Express 4.21, TypeScript 5, Sequelize 6.37 (PostgreSQL/MySQL), Mongoose 8.14 (MongoDB), Socket.IO 4.8, JWT (jsonwebtoken 9). Serviço de PLN (nlp-service) – Python, scikit-learn (TF-IDF + LinearSVC) para classificação de intenções e sentence-transformers (paraphrase-multilingual-MiniLM-L12-v2) para busca semântica, NLTK para pré-processamento/stemming em português. Integrações – Stripe (pagamentos), SendGrid (e-mail transacional), AWS S3/ImgBB (mídia), Expo Push (notificações), AWS Lambda (fallback de e-mail), Winston + Logtail/CloudWatch (logs).

4.3 Tecnologias Utilizadas

TypeScript 5.3+ (front e back), Node.js ≥ 18 , Express 4.21.2, Sequelize 6.37.8, Mongoose 8.14.1, PostgreSQL/MySQL, MongoDB, Socket.IO 4.8.3, React Native 0.81.5, Expo SDK 54, React 19.1, Zustand 5.0.5, React Navigation 7, Jest 29/30 + Supertest (testes), Docker/Docker Compose, GitHub Actions (CI/CD), Python 3 + scikit-learn + sentence-transformers + NLTK (nlp-service), Stripe, SendGrid, Winston.

5 MODELO DE DADOS

5.1 Diagrama de Entidade-Relacionamento (DER)

O modelo relacional (PostgreSQL, gerido por migrations Sequelize) tem User como entidade central, especializada em Client, Professional e Admin (1-para-1 por user_id). Um Professional oferece vários Service (agrupados por Category $\rightarrow$ Subcategory), define Availability e AvailabilityLock (bloqueios pontuais de agenda) e recebe Appointment de um Client em um Address. Cada Appointment pode gerar um ChatRoom (chat contextual) e grava rating/review quando avaliado. Favorite liga Client a Professional; Notification e ChatMessage ficam em MongoDB. O diagrama completo (entidades e FKs) deve ser exportado do próprio banco (ex.: sequelize-erd) ou desenhado no dbdiagram.io a partir da lista de entidades da seção 5.2 e anexado como imagem.

5.2 Descrição das Entidades Principais

User: id (PK), name, email (único), phone (único), password (hash bcrypt), active, avatar_uri. Client: id (PK), user_id (FK), main_address_id (FK), cpf (único). Professional: id (PK), user_id (FK), main_address_id (FK), cpf (único), cnpj (opcional), description, raio de atuação. Address: id (PK), user_id (FK), street, number, neighborhood, city, state, postal_code,

lat, lng. Category: id (PK), title, description, active. Subcategory: id (PK), title, category_id (FK), active. Service: id (PK), title, description, price (decimal), duration (min), subcategory_id (FK), professional_id (FK), active. ProfessionalAvailability/ServiceAvailability: id (PK), professional_id/service_id (FK), day_of_week (0-6), start_time, end_time. ProfessionalAvailabilityLock: id (PK), professional_id (FK), lock_date, start_time, end_time, reason. Appointment: id (PK), professional_id, client_id, service_id, address_id (FKs), start_time, end_time, status (pending/confirmed/completed/canceled), final_price, rating (1-5), review, payment_intent_id. Favorite: id (PK), user_id, professional_id (FKs). ChatRoom (Postgres): id (PK), appointment_id, professional_id, client_id, service_id (FKs), status. ChatMessage/Notification: coleções MongoDB (_id ObjectId), referenciando os IDs relacionais.

6 APIS (APPLICATION PROGRAMMING INTERFACES)

6.1 Design da API

API RESTful (JSON sobre HTTPS), documentada via Swagger/OpenAPI em /docs, complementada por WebSocket (Socket.IO) e Server-Sent Events (catálogo de serviços, GET /api/services/events). Principais grupos de rotas (base /api, exceto /auth): auth (POST /auth/register, /verify, /resend), user (POST /api/user/login, GET/PUT /api/user/me, POST /api/user/change-password), address (CRUD), categories/subcategories (leitura pública), professionals (busca com filtros de termo/geolocalização, cadastro/edição), services (CRUD completo, incluindo GET /api/services/my), professionalAvailability e availability-locks (CRUD de agenda), appointments (criar, listar, PATCH /:id/confirm, PUT /:id status, POST /:id/review, GET /:id/receipt), payments (create-payment-intent e confirm via Stripe), favorites (GET/POST/DELETE), notifications (listar/marcar lidas, salvar token Expo), dashboard (KPIs, ganhos, serviços por categoria), chat (GET /rooms, GET /rooms/:roomId/messages), admin (login e estatísticas). Já implementados na branch stag-facul (base usada para as entregas da faculdade, ainda não mesclada à main de produção): chatbot com PLN (POST /api/chat/bot/message, GET /api/chat/bot/session/active e /:id) e comando de voz/busca semântica (POST /api/voice/transcriptions, POST /api/voice/commands, GET /api/services/search/semantic).

6.2 Autenticação e Autorização

Autenticação stateless via JWT (HS256), emitido no login (/api/user/login e /api/admin/login) e enviado em Authorization: Bearer <token> em toda requisição privada. A

assinatura e a verificação são centralizadas (src/utils/jwt.util.ts) e o servidor não sobe sem a variável SECRET_KEY (o antigo valor padrão "secret" foi removido). O middleware auth.middleware.ts valida o token e injeta req.user/req.client/req.address; admin.middleware.ts restringe rotas administrativas. A autorização por recurso é feita nos services: cada operação confere se o usuário do token é dono do agendamento, endereço, serviço, disponibilidade ou notificação (evitando IDOR). Estão ativos helmet, hpp, mongo-sanitize, rate limit global e limitadores específicos para login (conta apenas tentativas falhas), cadastro, chatbot e voz, com trust proxy para identificar o IP real atrás do App Service. A sanitização de XSS e a detecção de SQL injection por regex foram removidas por corromperem senhas, URLs e mensagens do chatbot; a proteção efetiva vem das queries parametrizadas do Sequelize e do escape de dados em templates HTML de e-mail. Senhas usam hash bcrypt (inclusive no cadastro pendente de verificação), o login responde da mesma forma para e-mail inexistente e senha errada, o código de verificação de e-mail é gerado com gerador criptográfico e bloqueado após 5 tentativas, e o log de login no MongoDB guarda apenas uma impressão digital SHA-256 do token, nunca o JWT. No pagamento, o valor é calculado no servidor a partir do preço do serviço e o PaymentIntent fica vinculado ao usuário que o criou; a confirmação é idempotente.

7 COMPUTAÇÃO EM NUVEM (AZURE)

7.1 Serviços Azure Utilizados

O backend, que rodava em uma instância AWS EC2 (Docker) com deploy via GitHub Actions (build → Docker Hub → SSH deploy), está sendo migrado para a Azure. Já estão provisionados: um Resource Group (rg-delbicos); um App Service Plan Linux no tier Free (F1) (asp-delbicos-free), hospedando duas Web Apps publicadas sem contêiner — delbicos-backend-edu (API Node.js, runtime nativo NODE 20-lts) e delbicos-nlu (nlp-service Python, runtime nativo PYTHON 3.12, iniciado via uvicorn); e um Azure Key Vault (kv-delbicos), com Managed Identity nos App Services e acesso de leitura via a role Key Vault Secrets User. Ainda estão pendentes de provisionamento: Azure Database for PostgreSQL – Flexible Server; Azure Cosmos DB (API MongoDB) para chat e logs; Azure Blob Storage para mídia; Azure Static Web Apps para o build web do Expo; Application Insights + Azure Monitor para observabilidade. Não se usa Azure Container Registry nem Container Apps — o tier Free do App Service não aceita contêiner customizado, então o deploy é nativo (sem Docker). Para transcrição de voz, o backend passou a suportar o Azure Speech Service (provedor "azure", chave e região via variáveis de ambiente/Key Vault), com fallback automático para o Deepgram caso o Azure falhe; o suporte ao Gemini foi removido do projeto.

7.2 Arquitetura de Deploy

O deploy do backend no App Service é feito pelo workflow do GitHub Actions `deploy-azure-backend.yml`, disparado quando um Pull Request é mesclado na `stag-facul`: ele executa primeiro o workflow de testes (`typecheck + Jest + pytest`) e só então instala as dependências com `npm ci`, compila o TypeScript, remove as `devDependencies` e publica com a action `azure/webapps-deploy` (o pipeline anterior, Docker Hub + SSH para EC2, continua existindo para a branch `stag`). O `package-lock.json` passou a ser versionado para garantir builds reproduzíveis. Como o tier Free não suporta contêiner, tanto a API Node quanto o `nlp-service` são publicados como código-fonte; o `nlp-service` precisa de um comando de inicialização explícito, configurado como `uvicorn app.main:app --host 0.0.0.0 --port 8000`. O `requirements.txt` do `nlp-service` fixa a build `CPU-only` do PyTorch (via `extra-index-url` do próprio PyTorch), evitando baixar a variante com CUDA; o classificador de intenções (TF-IDF + SVM, sem dependência de PyTorch) é versionado já treinado em `artifacts/`, então não precisa ser retreinado a cada deploy — apenas o modelo de embeddings da busca semântica é baixado sob demanda, no primeiro uso real. O caminho do modelo (`MODEL_PATH`) é calculado relativo à posição do próprio código, funcionando tanto no Docker quanto no App Service. As migrations do Sequelize continuam previstas para rodar como um job único antes do rollout de cada nova versão, quando o PostgreSQL for provisionado. Os segredos não ficam em variáveis de ambiente em texto puro: os App Services usam Key Vault references (`@Microsoft.KeyVault(SecretUri=...)`), resolvidas via Managed Identity. O frontend web (build Expo Web) continua proposto como Azure Static Web App, ainda não provisionado. Como o `nlp-service` (`delbicos-nlu`) é acessível pela internet, suas rotas de negócio (`/classify`, `/semantic-search`) exigem uma chave de API compartilhada com o backend (header `X-API-Key`, segredo `NLU-SERVICE-API-KEY` no Key Vault), validada em tempo constante e com falha fechada; o endpoint `/health` permanece sem autenticação, por expor apenas o status do serviço.

7.3 Monitoramento e Logs

Já existe logging estruturado com Winston, exportado para Logtail e opcionalmente AWS CloudWatch. A centralização em Application Insights (requisições HTTP, exceptions, dependências como Stripe/SendGrid/`nlp-service`) e Azure Monitor (métricas de CPU/memória/latência e alertas, por exemplo taxa de erro 5xx acima de 5% em 5 minutos) continua proposta, mas ainda não foi provisionada.

7.4 Estimativa de Custos

Estimativa mensal aproximada (região Brazil South, uso baixo/MVP acadêmico, a validar na calculadora oficial da Azure), atualizada conforme os recursos provisionados até o momento: App Service Plan Free (F1), compartilhado entre a API Node e o nlp-service $\approx$ US\$ 0 (sujeito a mudar para Basic B1 caso o nlp-service estoure a cota de armazenamento de 1GB do tier Free); PostgreSQL Flexible Server Burstable B1ms $\approx$ US\$ 15 (ainda não provisionado); Blob Storage (poucos GB) $\approx$ US\$ 1–3 (ainda não provisionado); Static Web App (frontend web) – camada gratuita (ainda não provisionado); Application Insights – camada gratuita/baixo custo (ainda não provisionado); Key Vault – custo desprezível (já provisionado). Total estimado quando todos os recursos estiverem no ar: $\approx$ US\$ 16–20/mês para um ambiente de demonstração, podendo chegar a $\approx$ US\$ 29–35/mês caso o nlp-service precise migrar para o tier Basic.

8 QUALIDADE E TESTES DE SOFTWARE

8.1 Estratégia de Testes

Testes unitários por camada: regras de negócio puras (arquivos *.rules.ts e helpers) testadas diretamente, inclusive valores limite (antecedência mínima de 48 h, nota de 1 a 5, limite de 500 caracteres na avaliação, raio de atuação, CPF/CNPJ); services de domínio testados com os models Sequelize mockados, cobrindo autorização (ex.: impedir que um usuário leia ou altere agendamentos, endereços ou notificações de outro), transições de status do agendamento, idempotência e estorno do pagamento; e testes da aplicação Express com Supertest (headers de segurança, 404, JSON malformado, rotas protegidas). No frontend, stores Zustand, camada src/api e regras puras do chatbot são testadas com o cliente HTTP mockado. Testes manuais/exploratórios cobrem os fluxos ponta a ponta no app antes de cada entrega, complementados por testes de caixa preta (Partição de Equivalência/Análise de Valor Limite) e pelo diagrama de estados do agendamento (RF05), conforme exigido pela disciplina satélite de Qualidade e Testes de Software.

8.2 Ferramentas de Teste

Backend – Jest 29 + ts-jest + Supertest, com mock compartilhado da conexão de banco (src/config/__mocks__/database.ts), de modo que os testes rodam sem PostgreSQL/MongoDB; pytest no nlp-service. Frontend – jest-expo 54 com mock do AsyncStorage (jest.setup.js). CI – workflow "Testes" (tests.yml) no GitHub Actions dos dois repositórios, executado a cada push e pull request: typecheck, lint (frontend), Jest com relatório de cobertura publicado como artefato e pytest do nlp-service (backend). O deploy no Azure depende desse workflow passar.

8.3 Cobertura de Testes

Medição em 24/09/2026 na branch refactor/estrutura-seguranca (derivada da stag-facul), com a cobertura calculada sobre todo o código-fonte (collectCoverageFrom), e não apenas sobre os arquivos importados pelos testes — critério que antes inflava o número. Backend: 508 testes em 30 suítes, todos passando; cobertura de 71,9% das linhas e 73,3% dos ramos (os services de domínio refatorados, como appointment, catalog, professional, auth e dashboard, têm testes dedicados). nlp-service: 14 testes pytest passando. Frontend: 84 testes passando; cobertura de 24,7% da camada de lógica (stores, api, lib, hooks, utils) e de 8,5% do total, já que telas e componentes visuais somam cerca de 29 mil das ~36 mil linhas e são validados por testes manuais. Meta: 80% para o backend, exigido pela disciplina de Qualidade e Testes de Software; no frontend, ampliar a cobertura da camada de lógica, que ainda tem hooks (useChatSession) e parte das stores sem testes.

9 PROCESSAMENTO DE LINGUAGEM NATURAL (PLN)

9.1 Funcionalidades de PLN

(a) Chatbot de agendamento por texto — máquina de estados (BotSessionManager) que usa um classificador de intenção para entender pedidos de agendar, consultar, cancelar ou reagendar um serviço, mantendo o contexto da conversa e acionando as mesmas regras de negócio do restante do sistema (sem IA generativa tomando ações diretamente). (b) Comando de voz — a fala é transcrita para texto (Speech-to-Text) e enviada ao mesmo chatbot, reaproveitando toda a lógica de agendamento. (c) Busca semântica de serviços — encontra serviços por significado (ex.: “preciso aparar meu cabelo” → “corte de cabelo”) mesmo sem correspondência exata de palavras.

9.2 Tecnologias e Modelos

Um microserviço Python (nlp-service) concentra o PLN. Pré-processamento com NLTK (normalização, remoção de acentos/URLs/menções, tokenização e stemming com SnowballStemmer em português). Classificação de intenção com TF-IDF (uni/bigramas de palavras + n-gramas de caracteres) + LinearSVC (scikit-learn); a margem de decisão da SVM define quando retornar FALLBACK. Busca semântica com embeddings do modelo multilíngue paraphrase-multilingual-MiniLM-L12-v2 (sentence-transformers) e similaridade de cosseno. Datas, horários, IDs de agendamento e nomes de serviço são extraídos por regras determinísticas no backend Node (não é NER estatístico). O comando de voz usa o Azure Speech Service como provedor de transcrição, com fallback automático para o Deepgram; o

backend também aceita um provedor compatível com a API de transcrição da OpenAI (endpoint configurável) e um provedor mock para desenvolvimento.

9.3 Dataset e Treinamento

O classificador de intenções é treinado com um corpus próprio (intents.json) de frases de exemplo por intenção (AGENDAR, ALTERAR, CANCELAR, CONSULTAR, SAUDACAO e FALLBACK), curado pela equipe a partir de mensagens esperadas de clientes reais do DelBicos, complementado por regras explícitas para comandos curtos e inequívocos (ex.: “quero agendar”, “reiniciar”). O treino usa divisão estratificada de 80% treino / 20% teste (random_state=42) e publica as métricas em artifacts/metrics.json: no treino de 18/09/2026, com 285 amostras (57 no conjunto de teste), a acurácia foi de 98,25% e o F1 macro de 98,35%; a menor revocação foi em ALTERAR (88,9%) e a menor precisão em CONSULTAR (92,3%). O arquivo registra também a versão do modelo e o hash do corpus, permitindo rastrear qual versão está em produção. Em produção, o limiar de confiança (NLU_CONFIDENCE_THRESHOLD, padrão 0,65) e a taxa de FALLBACK observada em conversas de teste orientam ajustes no corpus.

10 INTEGRAÇÃO ENTRE COMPONENTES

10.1 Fluxo de Dados

App (mobile/web) → REST/JWT + Socket.IO → API Express (Node/TS) → PostgreSQL (dados relacionais) e MongoDB (chat e logs); a API Node consulta o nlp-service (Python) via HTTP para classificar intenção ou calcular similaridade semântica, autenticando-se com a chave X-API-Key (o serviço é acessível pela internet, mas recusa chamadas sem a chave). Pagamentos: o app solicita o PaymentIntent (a API calcula o valor pelo preço do serviço) → Stripe → confirmação na API, que cria ou vincula o Appointment de forma idempotente e estorna automaticamente se não conseguir gravá-lo. Mudanças de status de agendamento disparam eventos Socket.IO para os dispositivos conectados do cliente e do profissional, e o catálogo de serviços é atualizado no app via SSE.

10.2 Contratos de Interface

JSON sobre HTTP/HTTPS para a API REST; WebSocket (Socket.IO) para eventos em tempo real (chat, status de agendamento); SSE (text/event-stream) para atualizações do catálogo de serviços. Os contratos de payload seguem os tipos TypeScript compartilhados por

convenção entre frontend e backend (src/interfaces, src/types) e são documentados via Swagger/OpenAPI (/docs) para os endpoints REST.

11 RISCOS E MITIGAÇÕES

11.1 Identificação de Riscos

(1) A integração do reconhecimento de voz/PLN com a API de agendamento pode gerar falsos positivos de intenção, levando a agendamentos indesejados. (2) A stag-facul (base da entrega) ainda diverge da main de produção, e as refatorações de estrutura e segurança estão na branch refactor/estrutura-seguranca, ainda não mesclada na stag-facul; a promoção exigirá merge cuidadoso. (3) Riscos de segurança identificados na auditoria de 24/09/2026 e já corrigidos nessa branch: rotas sem autenticação (notificações, agendamentos por usuário, endereços), acesso a dados de outros usuários (IDOR), valor do pagamento definido pelo cliente, confirmação de pagamento com usuário vindo do corpo da requisição, exposição de CPF/e-mail em rotas públicas e JWT gravado nos logs. (4) A cobertura de testes do frontend ainda está abaixo da meta (ver seção 8.3). (5) Dependência de provedores externos de transcrição de voz (custo e disponibilidade). (6) Prazo curto do semestre para concluir a migração para a Azure sem interromper o ambiente atual (AWS EC2). (7) O nlp-service é acessível publicamente pela internet, sem isolamento de rede. (8) O segredo de assinatura dos tokens (SECRET_KEY) em uso tem menos de 32 caracteres; a equipe optou por mantê-lo por ora, pois a troca desconecta todos os usuários, e o servidor registra um aviso na inicialização.

11.2 Plano de Mitigação

(1) Manter sempre uma confirmação explícita do usuário antes de efetivar qualquer agendamento/cancelamento originado por voz ou chatbot (já implementado no fluxo de estados do bot). (2) Tratar a stag-facul como a branch de integração oficial até a entrega, mesclar a refactor/estrutura-seguranca por Pull Request revisado (o merge dispara o deploy no Azure somente após os testes passarem) e planejar depois um merge revisado para main. (3) Correções aplicadas com testes automatizados que reproduzem cada falha (ex.: listar agendamentos de outro usuário retorna 403; pagamento de terceiro retorna 403; confirmação repetida não duplica o agendamento). (4) Ampliar testes da camada de lógica do frontend e manter o workflow de testes obrigatório no CI. (5) Fallback automático Azure Speech → Deepgram e degradação graciosa para entrada por texto quando a voz falhar. (6) Migrar para Azure em paralelo ao ambiente atual (blue-green), trocando o domínio só após validação completa. (7) Chave de API compartilhada (header X-API-Key) nas rotas /classify e /semantic-search do nlp-service,

comparada em tempo constante e com falha fechada quando a chave não está configurada; o endpoint /health permanece público apenas para diagnóstico. (8) Gerar um novo segredo de 32+ caracteres (ex.: openssl rand -hex 32) e armazená-lo no Key Vault antes de ir para produção real.

12 ITENS DO PRODUCT BACKLOG (PBI)

Os itens do Product Backlog (RF01–RF10 e RNFs correspondentes) estão listados nas seções 3.1 e 3.2 deste documento, na ordem de prioridade definida pela equipe (Gestão de Bicos e Agenda como prioridade mais alta, seguidos de Chat, Segurança e Relatórios). A versão original do backlog está no arquivo docs/RF_RNF.pdf; as principais diferenças em relação à versão inicial são a adição das funcionalidades de PLN/voz e do chat em tempo real, incorporadas para atender à disciplina satélite de Processamento de Linguagem Natural.

13 ANEXOS

13.1 Backlog do Produto

docs/RF_RNF.pdf (tabela oficial de Requisitos Funcionais e Não Funcionais) e backend/docs/RF03_RF04_PLAN.md (plano de implementação do CRUD de serviços e do filtro por raio de atuação), no repositório do projeto.

13.2 Outros Diagramas

Recomenda-se anexar aqui: diagrama de casos de uso (Cliente, Profissional, Admin), diagrama de sequência do fluxo de agendamento + pagamento, diagrama de estados do agendamento (RF05, também exigido pela disciplina de Qualidade e Testes) e o DER exportado do banco (seção 5.1).

13.3 Referências

Documentação oficial Express.js (expressjs.com), Sequelize (sequelize.org), React Native (reactnative.dev), Expo (docs.expo.dev), Stripe API (stripe.com/docs/api), scikit-learn (scikit-learn.org), Sentence-Transformers (sbert.net); documentação interna do projeto: docs/CHATBOT_PLN.md, docs/DOCUMENTACAO_TECNICA_PLN_CHATBOT.md e docs/MVP_COMANDO_VOZ_BUSCA_SEMANTICA_BACKEND.md (branch stag-facul).